Universidad Nacional Autónoma de México
Facultad de Ingeniería

Bases de Datos

Tarea 8

Semestre: 2026-2
Profesor: Ing. Fernando Arreola

Alumno:
Parra Bello Carlos Enrique
No. Cuenta: 321135344

Grupo: 01

1. Tipos de datos en PostgreSQL

PostgreSQL cuenta con una gran variedad de tipos de datos nativos para almacenar información de manera precisa y eficiente. Elegir correctamente el tipo de dato permite ahorrar espacio, mejorar la integridad de los datos y facilitar las consultas. En esta tarea se describen los tipos numéricos, de carácter y de fecha, además de tres tipos adicionales que resultan especialmente interesantes por su utilidad en aplicaciones modernas.

1.1. Tipos numéricos

Los tipos numéricos se utilizan para almacenar cantidades enteras o con decimales. La elección depende del rango de valores que se desea guardar y de si se necesita exactitud o aproximación.

a) `smallint`

Capacidad: 2 bytes.
Rango: de `-32768` a `32767`.
Formato de representación: entero con signo de 16 bits.
Ejemplo:

```
edad smallint
```

Uso recomendado: valores enteros pequeños, como edades, cantidades bajas o indicadores numéricos simples.

b) `integer`

Capacidad: 4 bytes.
Rango: de `-2147483648` a `2147483647`.
Formato de representación: entero con signo de 32 bits.
Ejemplo:

```
stock integer
```

Uso recomendado: cantidades enteras de uso general.

c) `bigint`

Capacidad: 8 bytes.
Rango: de `-9223372036854775808` a `9223372036854775807`.
Formato de representación: entero con signo de 64 bits.
Ejemplo:

```
poblacion bigint
```

Uso recomendado: conteos muy grandes, identificadores numéricos extensos o acumulados masivos.

d) `numeric(p,s)`

Capacidad: variable.
Rango: hasta 131072 dígitos antes del punto decimal y hasta 16383 después.
Formato de representación: número exacto de precisión arbitraria.
Ejemplo:

```
precio numeric(10,2)
```

Uso recomendado: dinero, promedios exactos, cantidades contables y valores donde no se aceptan errores de redondeo.

e) `real` **y** `double precision`

Capacidad: 4 bytes y 8 bytes, respectivamente.
Precisión aproximada: 6 dígitos decimales para `real` y 15 para `double precision`.
Formato de representación: punto flotante.
Ejemplo:

```
temperatura real,
promedio double precision
```

Uso recomendado: cálculos científicos o mediciones donde se acepta una aproximación.

1.2. Tipos de carácter

Estos tipos se emplean para almacenar texto, nombres, descripciones, correos, direcciones y cualquier cadena de caracteres.

a) `character varying(n)` **o** `varchar(n)`

Capacidad: variable, con un límite de `n` caracteres.
Formato de representación: cadena de longitud variable.
Ejemplo:

```
nombre varchar(80)
```

Uso recomendado: nombres, correos electrónicos, títulos o textos cortos con longitud máxima conocida.

b) `character(n)` **o** `char(n)`

Capacidad: fija, exactamente `n` caracteres.
Formato de representación: cadena de longitud fija, rellenada con espacios si es necesario.
Ejemplo:

```
sexo char(1)
```

Uso recomendado: códigos de longitud fija, como claves cortas o abreviaturas.

c) `text`

Capacidad: longitud variable, prácticamente hasta aproximadamente 1 GB.
Formato de representación: texto libre sin límite específico declarado.
Ejemplo:

```
descripcion text
```

Uso recomendado: comentarios, observaciones, contenido largo o descripciones extensas.

1.3. Tipos de fecha y hora

Permiten almacenar fechas, horas, marcas de tiempo e intervalos.

a) `date`

Capacidad: 4 bytes.
Formato de representación: fecha calendario (año-mes-día).
Ejemplo:

```
fecha_nacimiento date
```

Uso recomendado: cumpleaños, fechas de inscripción, entregas o registros de eventos.

b) `time`

Capacidad: 8 bytes.
Formato de representación: hora del día sin fecha.
Ejemplo:

```
hora_entrada time
```

Uso recomendado: horarios, turnos o tiempos específicos dentro de un día.

c) `timestamp`

Capacidad: 8 bytes.
Formato de representación: fecha y hora juntas.
Ejemplo:

```
fecha_registro timestamp
```

Uso recomendado: auditorías, creación de registros y seguimiento detallado de eventos.

d) `timestamp with time zone`

Capacidad: 8 bytes.
Formato de representación: fecha y hora con zona horaria.
Ejemplo:

```
fecha_evento timestamp with time zone
```

Uso recomendado: sistemas distribuidos o aplicaciones internacionales.

e) `interval`

Capacidad: 16 bytes.
Formato de representación: intervalo de tiempo.
Ejemplo:

```
duracion interval
```

Uso recomendado: duración de procesos, tiempos de espera, diferencias entre fechas y periodos.

1.4. Otros tres tipos interesantes

1) `uuid`

El tipo `uuid` almacena identificadores universales únicos de 128 bits. Es muy útil cuando se desea que un identificador sea prácticamente irrepetible, especialmente en sistemas distribuidos. Su formato típico consiste en 32 dígitos hexadecimales separados por guiones.

Capacidad: 128 bits.
Formato de representación:

```
a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11
```

Ejemplo:

```
id_usuario uuid
```

Uso recomendado: llaves primarias globales, microservicios, sincronización entre bases de datos y aplicaciones distribuidas.

2) `json` **y** `jsonb`

Estos tipos permiten almacenar información en formato JSON. El tipo `json` guarda una copia textual del dato, mientras que `jsonb` lo almacena en formato binario descompuesto, lo que hace más eficiente su procesamiento e indexación.

Capacidad: variable.
Formato de representación:

```
{"nombre":"Carlos","edad":20,"activo":true}
```

Ejemplo:

```
perfil jsonb
```

Uso recomendado: configuraciones, documentos semiestructurados, respuestas de APIs y datos flexibles.

3) `array`

PostgreSQL permite que una columna almacene arreglos de una o varias dimensiones. Es una característica interesante porque facilita guardar colecciones pequeñas dentro de un mismo registro.

Capacidad: variable.

Formato de representación:

`{10,20,30,40}`

Ejemplo:

```
calificaciones integer[]
```

Uso recomendado: listas pequeñas de valores relacionados, horarios, etiquetas o datos agrupados.

1.5. Conclusión

Los tipos de datos en PostgreSQL ofrecen una gran flexibilidad para modelar información de forma correcta. Los tipos numéricos permiten elegir entre exactitud y rango; los de carácter son adecuados para almacenar desde códigos fijos hasta texto libre; y los de fecha facilitan el manejo de eventos temporales. Además, tipos como `uuid`, `jsonb` y `array` muestran que PostgreSQL no solo cubre necesidades tradicionales, sino también requerimientos modernos orientados a sistemas distribuidos, datos semiestructurados y estructuras más complejas. Por ello, seleccionar adecuadamente el tipo de dato es una decisión importante en el diseño de una base de datos.

Referencias

[1] PostgreSQL Global Development Group, "Chapter 8. Data Types," *PostgreSQL Documentation*, versión 18, 2026.

[2] PostgreSQL Global Development Group, "8.1. Numeric Types," *PostgreSQL Documentation*, versión 18, 2026.

[3] PostgreSQL Global Development Group, "8.3. Character Types," *PostgreSQL Documentation*, versión 18, 2026.

[4] PostgreSQL Global Development Group, "8.5. Date/Time Types," *PostgreSQL Documentation*, versión 18, 2026.

[5] PostgreSQL Global Development Group, "8.12. UUID Type," *PostgreSQL Documentation*, versión 18, 2026.

[6] PostgreSQL Global Development Group, "8.14. JSON Types," *PostgreSQL Documentation*, versión 18, 2026.

[7] PostgreSQL Global Development Group, "8.15. Arrays," *PostgreSQL Documentation*, versión 18, 2026.